

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: ITA 0606 COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO5: *Introduction – NumPy Arrays – NumPy Basics- Math – Random – Indexing – Filtering – Statistics – Aggregation – saving data – Data analysis with Pandas – DataFrame – Combining – Indexing – File I/O – Grouping – Filtering – Sorting – Plotting (BL 3)*

Assessment Tool Weightage: 40%

QUESTIONS: 5 Total Marks:100 Annexure A

Constraint Based Problem Solving

Code of Conduct:

I Prudhvi Raj.B(192324274) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:prudhvi raj.b

Annexure A

1. You are given a large dataset of patient health records stored in a CSV file, containing missing values, inconsistent formats, and outliers. Using NumPy and Pandas, design a constraint-based data cleaning pipeline that ensures: (i) no missing values remain, (ii) all numerical values fall within medically valid ranges, and (iii) categorical fields are standardized. Explain how you will use indexing, filtering, and aggregation functions to enforce these constraints while preserving maximum data integrity.
2. A system with limited memory must process a massive numerical dataset using NumPy arrays. You are required to perform mathematical operations, random sampling, and statistical analysis without exceeding memory limits. Propose a solution that uses efficient array creation, slicing, and in-place operations. Discuss how indexing and aggregation can be optimized to minimize memory usage, and explain debugging strategies if memory overflow or performance bottlenecks occur.
3. Two large Pandas DataFrames containing customer transaction and demographic data must be combined. However, constraints include: (i) no duplication of customer IDs, (ii) consistent indexing after merging, and (iii) preservation of all valid records. Describe how you will perform the combining operation, validate constraints using filtering and grouping, and ensure correctness through sorting and indexing techniques. Include steps to debug mismatched or missing entries.
4. You are tasked with computing statistical measures (mean, median, variance) on a dataset using NumPy and Pandas, but with constraints on both computational efficiency and numerical accuracy. Explain how you will implement aggregation and statistical functions while avoiding precision loss and redundant computations. Discuss how filtering and indexing can be used to exclude irrelevant data points, and how you would debug incorrect statistical outputs.
5. Design a complete data analysis pipeline that reads multiple data files, processes them using Pandas DataFrames, and generates visualizations. Constraints include: (i) efficient file I/O operations, (ii) correct grouping and sorting of data, (iii) filtering based on user-defined conditions, and (iv) meaningful plotting outputs. Explain how each stage (loading, processing, aggregation, and plotting) will be implemented and optimized, along with debugging steps for handling corrupted files or inconsistent data formats

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	15	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

1. Constraint-Based Data Cleaning Using NumPy and Pandas

Approach

A data-cleaning pipeline can be designed as follows:

Load the CSV file

```
import pandas as pd
import numpy as np
```

1. `df = pd.read_csv("patient_data.csv")`
2. **Identify missing values**
`print(df.isnull().sum())`
3. **Handle missing values**
 - Numerical columns → replace missing values with median.
 - Categorical columns → replace missing values with mode.

```
df["Age"] = df["Age"].fillna(df["Age"].median())
```

```
4. df["Gender"] = df["Gender"].fillna(df["Gender"].mode()[0])
```

Standardize categorical values

```
df["Gender"] = df["Gender"].str.strip().str.lower()
```

```
df["Gender"] = df["Gender"].replace({  
    "m": "male",  
    "f": "female"
```

```
5. })
```

Enforce medically valid ranges

```
df.loc[(df["Age"] < 0) | (df["Age"] > 120), "Age"] = np.nan
```

```
df.loc[(df["SystolicBP"] < 70) | (df["SystolicBP"] > 250), "SystolicBP"] = np.nan
```

```
6. df.loc[(df["Cholesterol"] < 50) | (df["Cholesterol"] > 500), "Cholesterol"] = np.nan
```

The resulting missing values can again be filled using appropriate statistical values or, when necessary, removed if they cannot be reliably recovered.

7. Use filtering and indexing

```
valid = df[(df["Age"] >= 0) & (df["Age"] <= 120)]
```

Use aggregation for validation

```
print(df.groupby("Gender")["Age"].mean())
```

```
8. print(df.describe())
```

Final constraint verification

```
print(df.isnull().sum())
```

```
9. print(df["Age"].between(0, 120).all())
```

Data Integrity

To preserve maximum integrity, original values should be backed up before modification. Instead of deleting valid records, invalid values should preferably be corrected or replaced using reliable rules.

Conclusion: NumPy provides efficient numerical operations, while Pandas provides indexing, filtering, grouping, and missing-value handling needed to create a constraint-based cleaning pipeline.

2. Memory-Efficient Processing of a Massive NumPy Dataset

Approach

For a massive dataset, memory-efficient NumPy operations should be used.

Create arrays with appropriate data types

import numpy as np

1. `arr = np.arange(10000000, dtype=np.float32)`
`float32` requires less memory than `float64` when its precision is sufficient.
2. **Use slicing instead of copying**
`subset = arr[1000:5000]`
Basic NumPy slicing generally creates a **view**, avoiding unnecessary copying.

Use in-place operations

`arr *= 2`

3. `arr += 10`
This avoids creating additional large arrays.
4. **Random sampling**
`sample = np.random.choice(arr, size=1000, replace=False)`

Statistical analysis

`mean = np.mean(arr)`

`median = np.median(arr)`

5. `variance = np.var(arr)`
6. **Use indexing efficiently**
`subset = arr[arr > 50]`
For very large arrays, avoid creating multiple intermediate arrays unnecessarily.

Aggregation

`total = np.sum(arr)`

`maximum = np.max(arr)`

7. `minimum = np.min(arr)`

Debugging Memory Problems

- Check memory usage with `arr.nbytes`.
- Use smaller data types when acceptable.
- Avoid unnecessary `.copy()`.
- Process data in chunks when the complete dataset cannot fit in RAM.
- Profile expensive operations to identify bottlenecks.
- Delete unused arrays using `del`.
- Use memory-efficient algorithms instead of repeated concatenation.

Conclusion: Efficient data types, slicing, in-place operations, aggregation, and chunk processing allow massive NumPy datasets to be processed without exceeding memory limits.

3. Combining Two Large Pandas DataFrames

Assume:

- `transactions` contains customer transaction information.
- `demographics` contains customer demographic information.
- `CustomerID` is the common key.

Approach

Load the DataFrames

import pandas as pd

```
transactions = pd.read_csv("transactions.csv")
```

```
1. demographics = pd.read_csv("demographics.csv")
```

Check duplicate customer IDs

```
print(transactions["CustomerID"].duplicated().sum())
```

```
2. print(demographics["CustomerID"].duplicated().sum())
```

```
3. Remove or resolve duplicate demographic records
```

```
demographics = demographics.drop_duplicates("CustomerID")
```

Merge the DataFrames

To preserve all valid transaction records:

```
merged = transactions.merge(  
    demographics,  
    on="CustomerID",  
    how="left",  
    validate="many_to_one"
```

```
4. )
```

```
5. Check missing customer IDs
```

```
missing = merged[merged["CustomerID"].isnull()]
```

Check unmatched demographic records

```
unmatched = demographics[  
    ~demographics["CustomerID"].isin(transactions["CustomerID"])]
```

```
6. ]
```

```
7. Use grouping and filtering
```

```
print(merged.groupby("CustomerID").size())
```

Set a consistent index

```
merged = merged.sort_values("CustomerID")
```

```
8. merged = merged.reset_index(drop=True)
```

Validate the result

```
print(merged.shape)
```

```
9. print(merged["CustomerID"].duplicated().sum())
```

Debugging

If records are missing:

- Check whether IDs have different data types.
- Remove leading/trailing spaces.
- Check capitalization or formatting differences.
- Compare IDs in both DataFrames.
- Check for duplicate keys.

Example:

```
transactions["CustomerID"] = transactions["CustomerID"].astype(str).str.strip()
demographics["CustomerID"] = demographics["CustomerID"].astype(str).str.strip()
```

Conclusion: A validated merge with consistent keys, duplicate checking, filtering, grouping, sorting, and index resetting ensures that all valid customer records are preserved correctly.

4. Computing Mean, Median and Variance Efficiently

NumPy Implementation

```
import numpy as np

data = np.array([10, 20, 30, 40, 50], dtype=np.float64)

mean = np.mean(data)
median = np.median(data)
variance = np.var(data)

print(mean)
print(median)
print(variance)
```

Pandas Implementation

```
import pandas as pd

df = pd.DataFrame({"Value": [10, 20, 30, 40, 50]})

print(df["Value"].mean())
print(df["Value"].median())
print(df["Value"].var())
```

Filtering Irrelevant Data

If values outside a valid range should be excluded:

```
valid = df[df["Value"].between(10, 50)]
```

Then calculate statistics:

```
mean = valid["Value"].mean()
median = valid["Value"].median()
variance = valid["Value"].var()
```

Improving Accuracy and Efficiency

- Use `float64` when greater numerical precision is required.
- Avoid repeatedly recalculating the same statistics.
- Remove invalid or irrelevant values before aggregation.
- Use vectorized NumPy/Pandas functions.
- Handle missing values appropriately.
- Use appropriate variance definition: Pandas `var()` uses sample variance by default, while NumPy `var()` uses population variance by default (`ddof=0`).

For sample variance in NumPy:

```
variance = np.var(data, ddof=1)
```

Debugging Incorrect Results

Check:

```
print(df.info())
print(df.isnull().sum())
print(df.describe())
print(df["Value"].unique())
```

Also verify:

- Number of observations.
- Data type.
- Missing values.
- Outliers.
- Filtering conditions.
- `ddof` setting for variance.

Conclusion: NumPy and Pandas aggregation functions provide accurate and efficient statistical calculations when data types, filtering, missing values, and variance definitions are handled correctly.

5. Complete Data Analysis Pipeline

Step 1: Load Multiple Files

```
import pandas as pd
import glob
```

```
files = glob.glob("data/*.csv")
```

```
df = pd.concat(  
    [pd.read_csv(file) for file in files],  
    ignore_index=True  
)
```

For large files, use only required columns:

```
df = pd.read_csv(  
    "data.csv",  
    usecols=["Date", "Category", "Sales"]  
)
```

This reduces memory usage.

Step 2: Inspect and Clean Data

```
print(df.head())  
print(df.info())  
print(df.isnull().sum())
```

Convert data types:

```
df["Date"] = pd.to_datetime(df["Date"], errors="coerce")  
df["Sales"] = pd.to_numeric(df["Sales"], errors="coerce")
```

Handle missing values:

```
df["Sales"] = df["Sales"].fillna(df["Sales"].median())
```

Step 3: Filter Data

User-defined conditions can be applied using Boolean filtering.

```
filtered = df[df["Sales"] > 1000]
```

Multiple conditions:

```
filtered = df[  
    (df["Sales"] > 1000) &  
    (df["Category"] == "Electronics")  
]
```

Step 4: Group and Aggregate

```
summary = (  
    filtered  
    .groupby("Category")["Sales"]  
    .sum()  
)
```

```
.reset_index()  
)
```

Step 5: Sort Results

```
summary = summary.sort_values(  
    "Sales",  
    ascending=False  
)
```

Step 6: Visualization

import matplotlib.pyplot as plt

```
plt.bar(summary["Category"], summary["Sales"])  
plt.xlabel("Category")  
plt.ylabel("Total Sales")  
plt.title("Sales by Category")  
plt.xticks(rotation=45)  
plt.show()
```

A line plot can be used for time-series data:

```
daily = df.groupby("Date")["Sales"].sum()
```

```
daily.plot(kind="line")  
plt.xlabel("Date")  
plt.ylabel("Sales")  
plt.title("Daily Sales Trend")  
plt.show()
```

Optimization

- Read only required columns using `usecols`.
- Specify appropriate data types using `dtype`.
- Process very large CSVs using `chunksize`.
- Avoid unnecessary DataFrame copies.
- Use vectorized operations.
- Group and aggregate only required columns.
- Sort only when sorting is actually required.

Debugging Corrupted/Inconsistent Files

Use:

try:

```
df = pd.read_csv("data.csv")  
except Exception as e:  
    print("File reading error:", e)
```

For inconsistent data:

```
df.columns = df.columns.str.strip()  
df["Category"] = df["Category"].str.strip().str.lower()
```

Check file structure:

```
print(df.shape)  
print(df.columns)  
print(df.dtypes)  
print(df.isnull().sum())
```

Result: The pipeline loads, cleans, filters, groups, sorts, aggregates, and visualizes data efficiently.

Conclusion: Combining Pandas for data processing with Matplotlib for visualization provides an efficient and reliable end-to-end data analysis workflow.